# Abstract

## Abstract I

The reproducibility problem in computational research is partly structural: research artifacts are often distributed across editors, notebooks, scripts, and manual publication steps without a shared mechanism that keeps code, data, and manuscript synchronized. Existing tools address important parts of this lifecycle: workflow managers orchestrate computation, literate-programming systems render documents, and data-versioning tools track artifacts. `template/` combines those concerns with test and provenance gates by applying Infrastructure as Code to the research lifecycle. Its Two-Layer Architecture separates 31 infrastructure subdirectories (28 importable Python packages, ~1355 modules, validated by ~13,093 tests) from self-contained project workspaces. A YAML-declared pipeline (17 stages; 10 in a default full run) coordinates environment checks, test

execution, analysis, Pandoc/XeLaTeX rendering, SHA-256 hashing with optional steganographic watermarking, structural validation, and optional LLM review. The declared quality floors—90% for project source and 60% for infrastructure source—are injected from executable configuration rather than copied into the manuscript. Documentation Duality equips repository directories with human-facing `README.md` and agent-facing `AGENTS.md` files; skill-enabled infrastructure packages additionally expose structured `SKILL.md` descriptors so agents can discover supported capabilities from versioned interfaces.

## Abstract I

The generated public exemplar roster (templates/template_active_inference, templates/template_advanced_literature_review, templates/template_autopoiesis, templates/template_autoresearch_project, templates/template_autoscientists, templates/template_code_project, templates/template_data_descriptor, templates/template_eda_notebook, templates/template_experiment_tree, templates/template_formal, templates/template_gold_refinement, templates/template_literature_meta_analysis, templates/template_madlib, templates/template_methods_paper, templates/template_newspaper, templates/template_pitch_deck, templates/template_pools_rules_tools, templates/template_prose_project, templates/template_redacted_report,

templates/template_registered_report, templates/template_search_project, templates/template_sia, templates/template_storybook, templates/template_template, templates/template_textbook) supplies heterogeneous control-positive layouts, including optimization (template_code_project, 258 discovered tests), prose (template_prose_project, 165 discovered tests), and AutoResearch readiness (template_autoresearch_project, 318 discovered tests). Each declares the same 90% project-source floor and consumes the shared pipeline without project-to-project imports. This manuscript adds a reflexive artifact: template_template (158 discovered tests) follows the same analysis and render path while injecting repository counters and figures from live introspection. The result is a self-documenting build whose quantitative repository claims

are generated artifacts, whose manuscript claims are checked against an evidence registry, and whose output can be validated and optionally watermarked. A versioned comparative matrix situates `template/` against peer tools across the capabilities reported in the Results section. Code is released under the Apache License 2.0 at `github.com/docxology/template`; the work remains open-ended.